

Azure Data Factory Setup Guide

Step-by-step walkthrough: creating storage containers, a Data Factory, and a pipeline that loops through files and branches with an If Condition.

STEP 1

Check the storage account containers

Opened the storage account (storage3354) and viewed its Containers list. Two containers already exist: \$logs and input.

Microsoft Azure

Upgrade

Search resources, services, and docs (G+I)

Copilot

Santo.Joseph@akumen...
AKUMENBYQ (AKUMENBYQ.COM)

Home > storage3354

storage3354

Storage account

Search

◊ <<

+ Add container

↑ Upload

↻ Refresh

🗑 Delete

🔒 Change access level

🔄 Restore containers

🔗 Edit columns

Overview

Activity log

Tags

Diagnose and solve problems

Access Control (IAM)

Data migration

Events

Storage browser

Storage Mover

Partner solutions

Resource visualizer

Data storage

Containers

Classic file shares

Queues

Tables

Search containers by prefix

Only show active containers

Showing all 2 items

☐	Name	Last modified	Anonymous access level	Lease state	Storage connector
☐	\$logs	24/08/2026, 13:57:42	Private	Available	-
☐	input	24/08/2026, 14:18:33	Private	Available	-

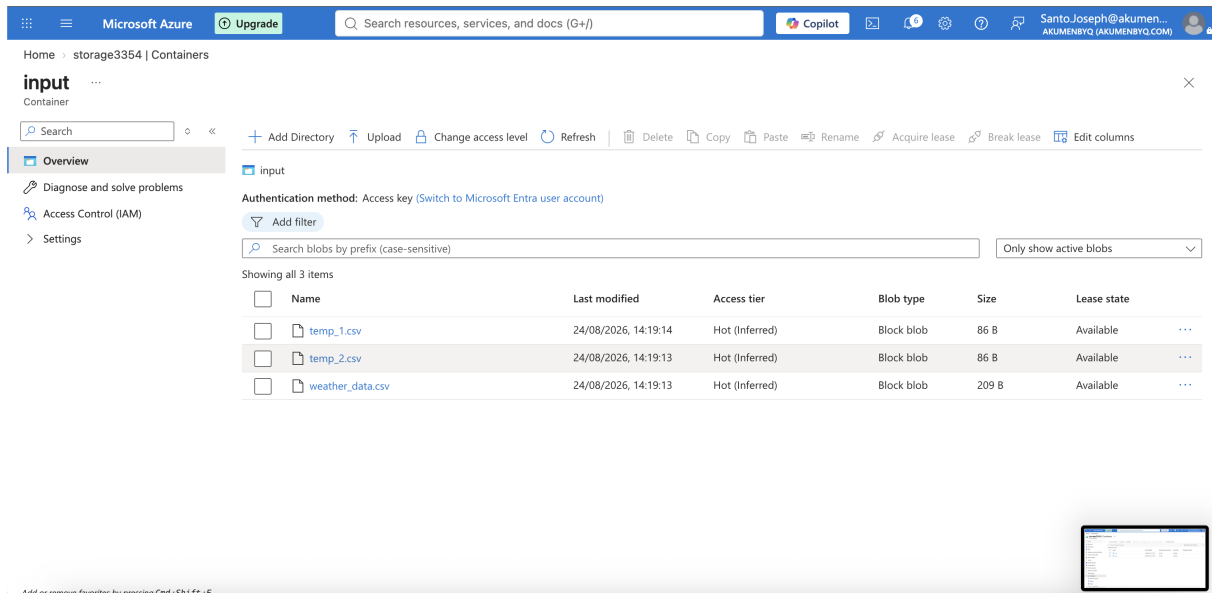
Carroll & nashovkinn

Add or remove favorites by pressing Cmd+Shift+F

STEP 2

Review files inside the "input" container

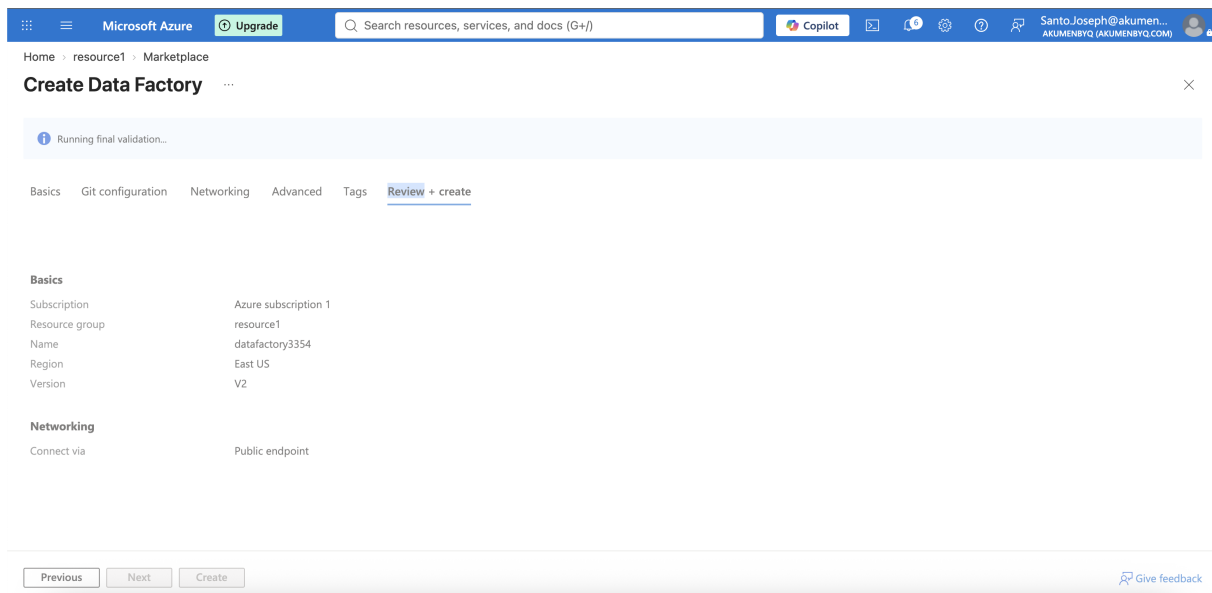
Opened the input container to confirm the source files are present: temp_1.csv, temp_2.csv, and weather_data.csv.



STEP 3

Review Data Factory settings before creating it

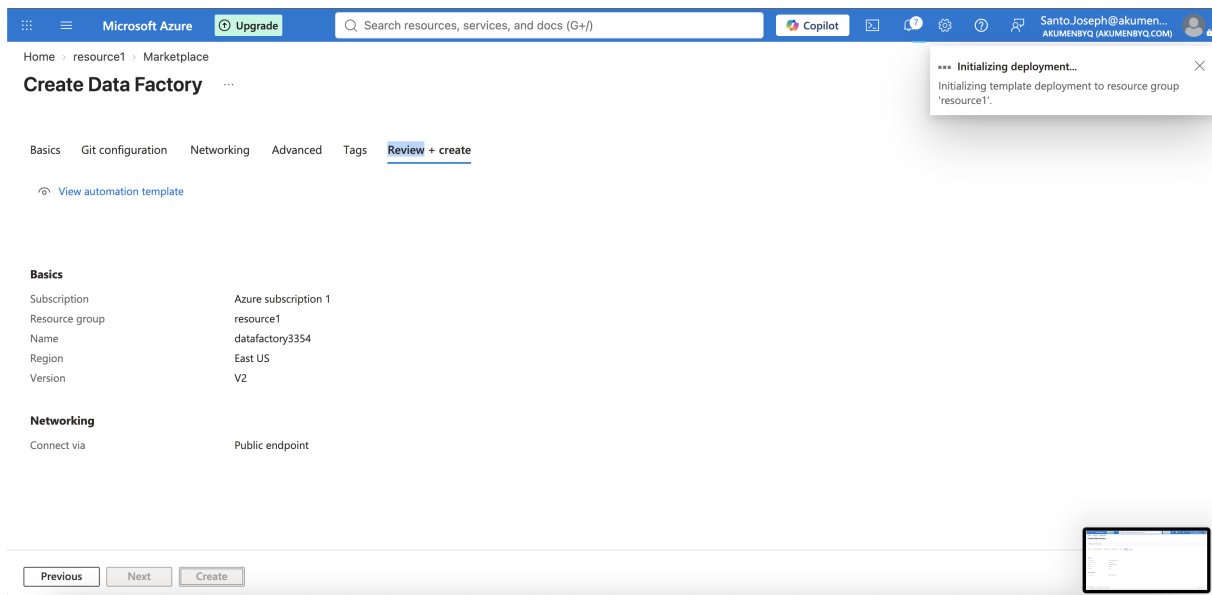
On the "Create Data Factory" screen, reviewed the settings under Review + create: Resource group (resource1), Name (datafactory3354), Region (East US), Version (V2), connected via Public endpoint.



STEP 4

Deployment starts

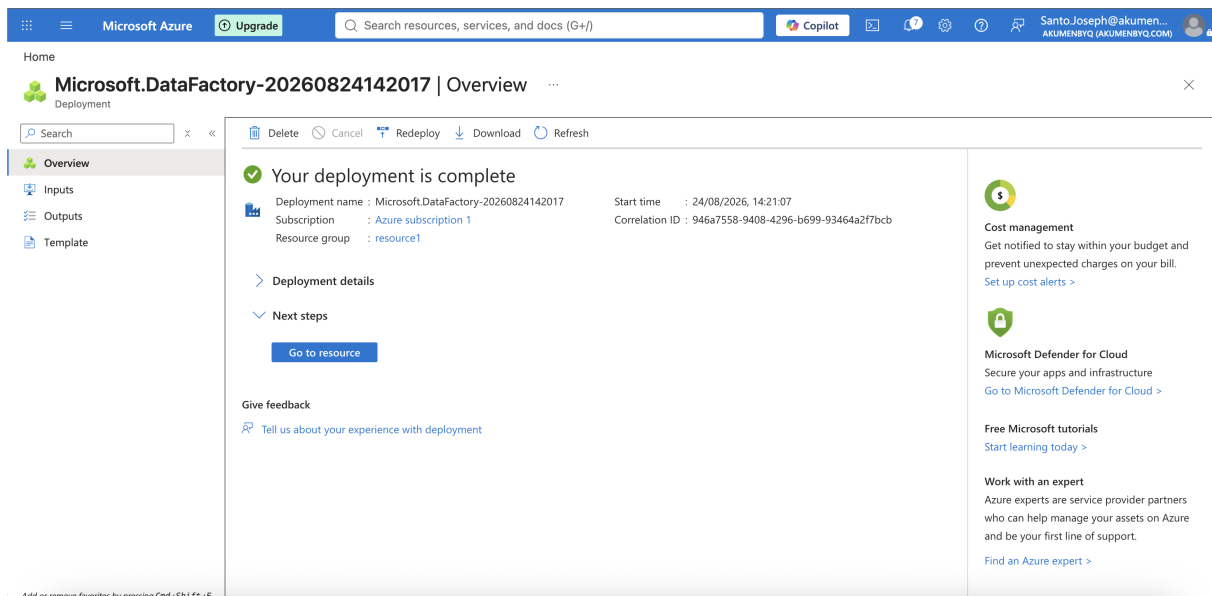
Clicked Create. Azure begins initializing the template deployment to the resource group "resource1".



STEP 5

Deployment complete

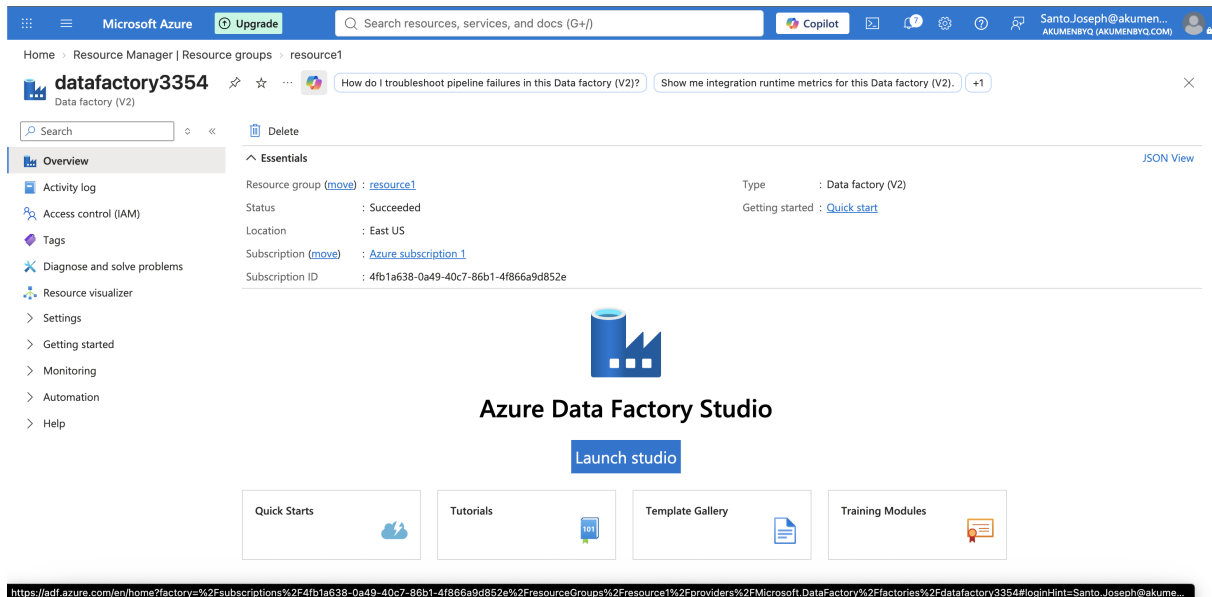
The Data Factory deployment finished successfully (Microsoft.DataFactory-20260824142017).



STEP 6

Open the new Data Factory

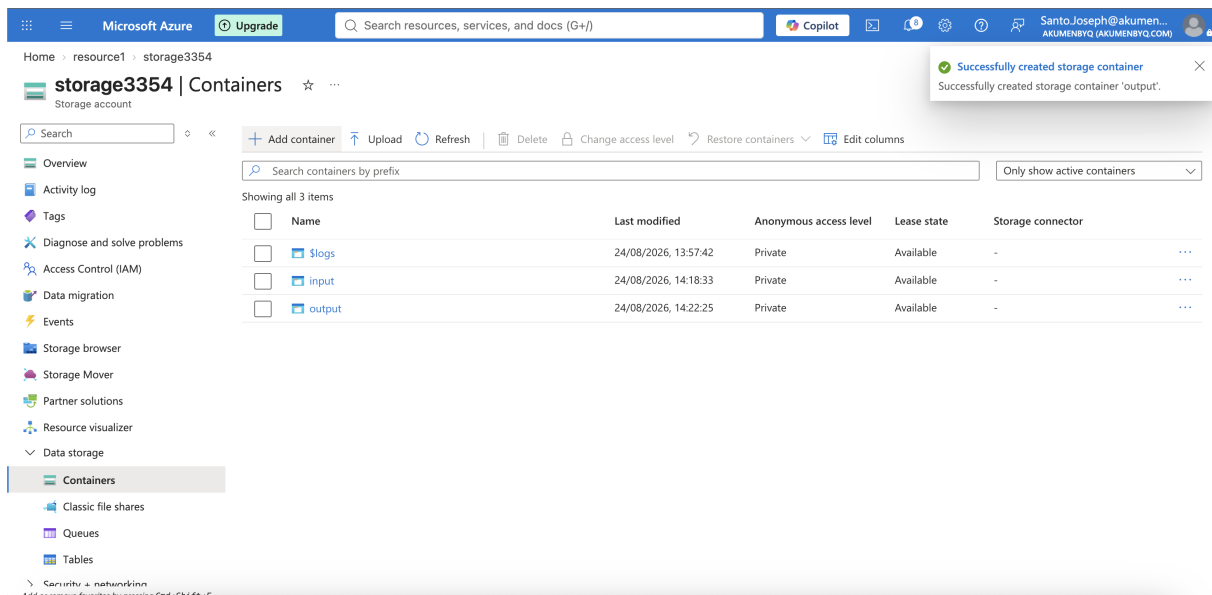
Opened the datafactory3354 resource (Type: Data factory V2, Status: Succeeded, Location: East US) and clicked "Launch studio" to open Azure Data Factory Studio.



STEP 7

Create an "output" container

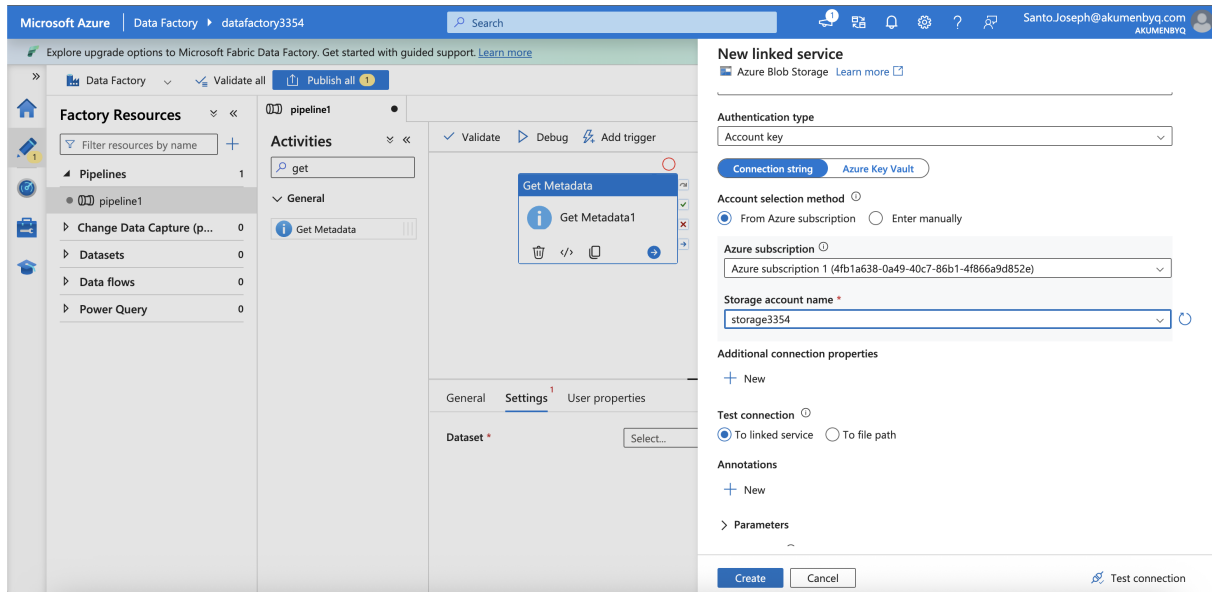
Back in the storage account, added a new container named "output" so the pipeline has a destination to write results to. The storage account now shows 3 containers: \$logs, input, output.



STEP 8

Create a Get Metadata activity and linked service

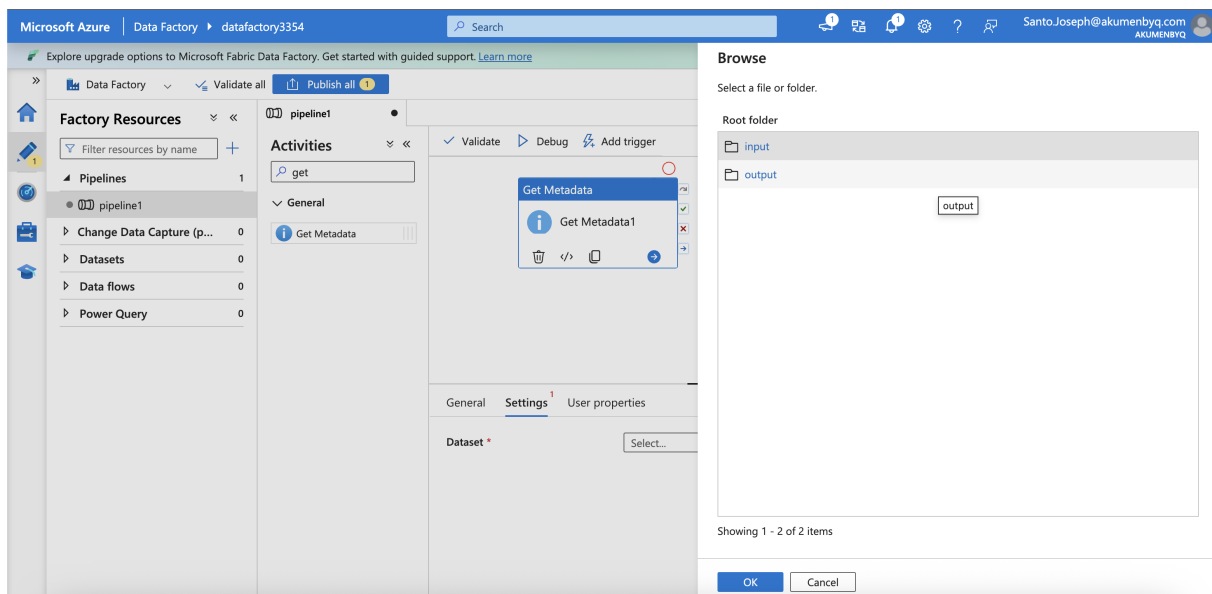
In the pipeline canvas, added a "Get Metadata" activity. While configuring its dataset, created a new Linked Service pointing to Azure Blob Storage, using Account key authentication and selecting the storage3354 account.



STEP 9

Browse and select the folder

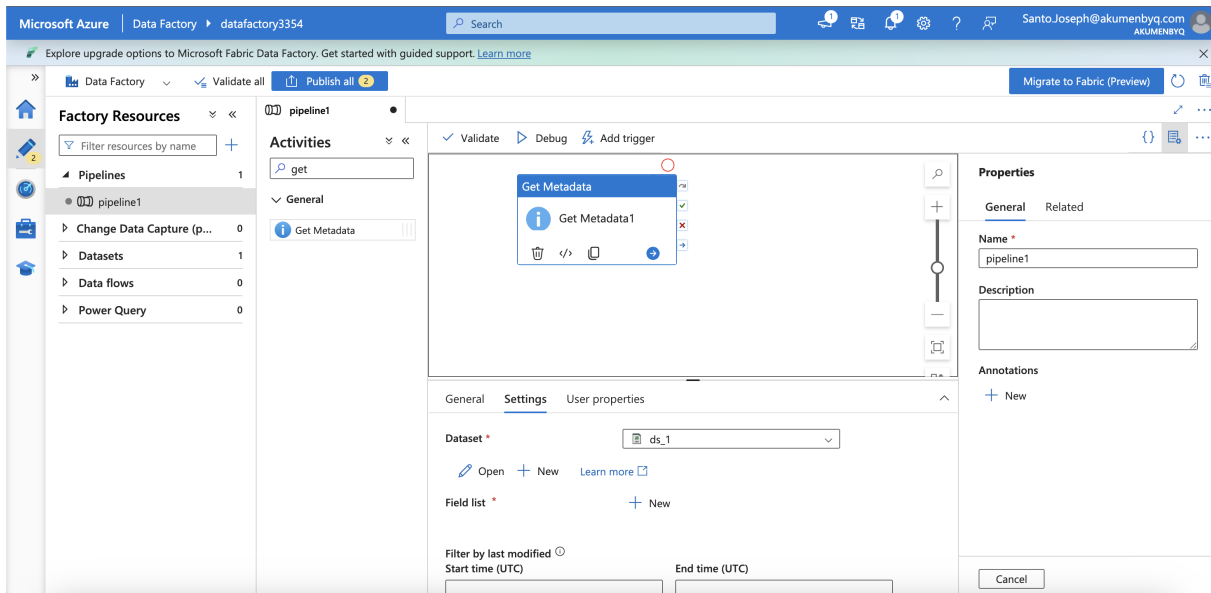
Used the Browse window to pick the folder the Get Metadata activity should scan, selecting the "output" folder from the root.



STEP 10

Finish configuring Get Metadata

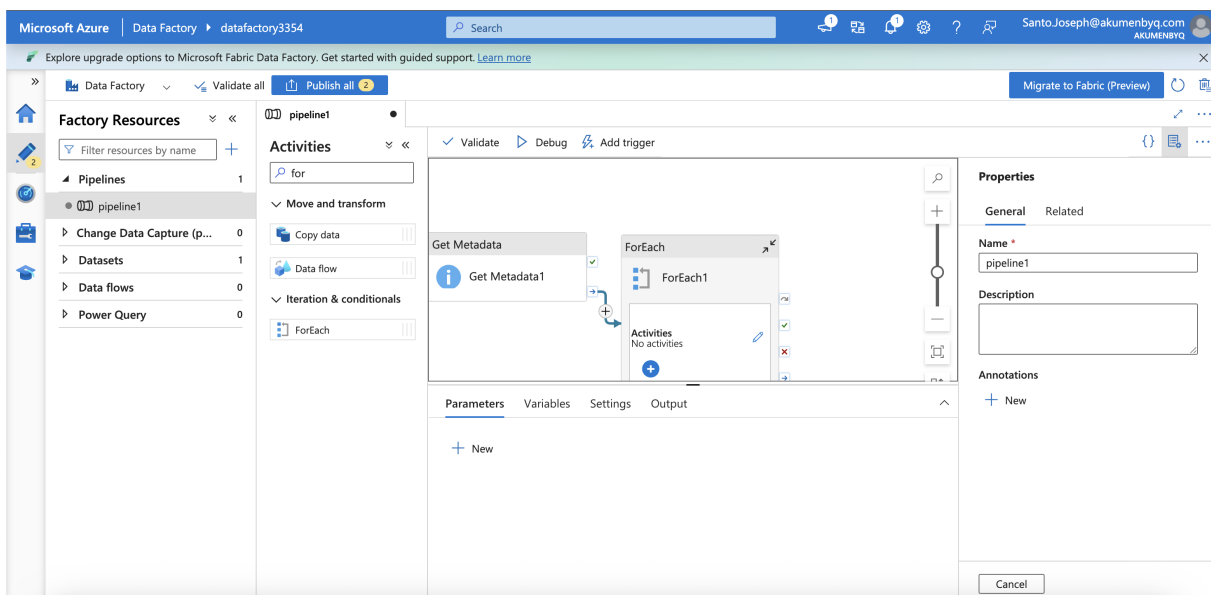
With the dataset (ds_1) linked, the Get Metadata1 activity is set up on the Settings tab, ready to have a Field list added.



STEP 11

Add a ForEach activity

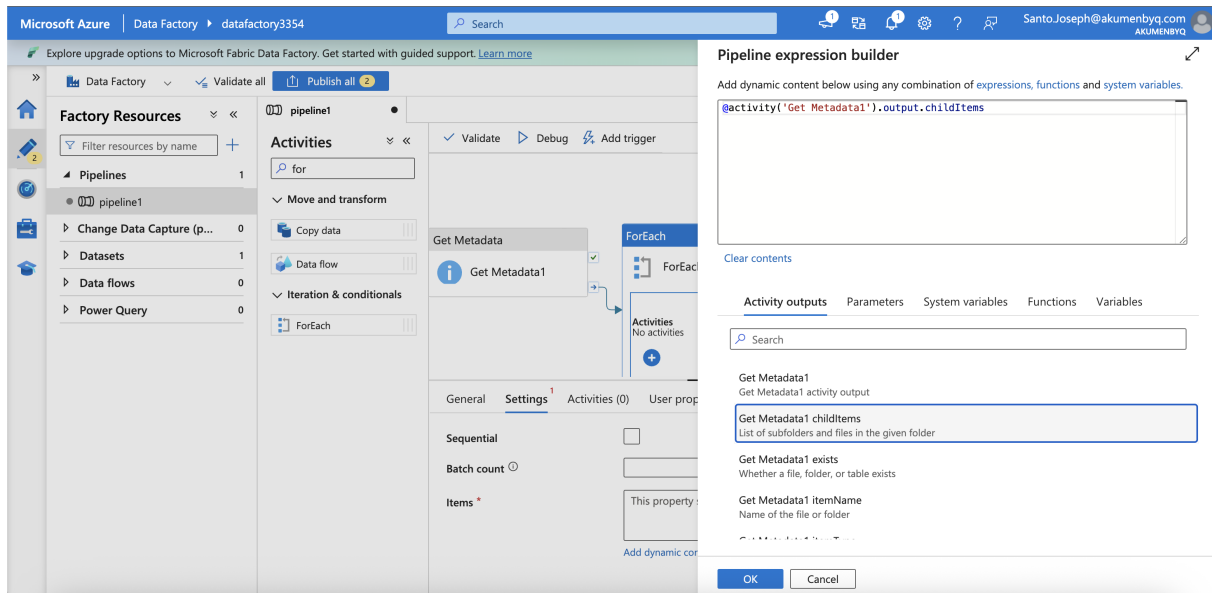
Added a "ForEach" activity and connected it after Get Metadata1, so the pipeline can loop through each item returned by Get Metadata.



STEP 12

Set the ForEach "Items" expression

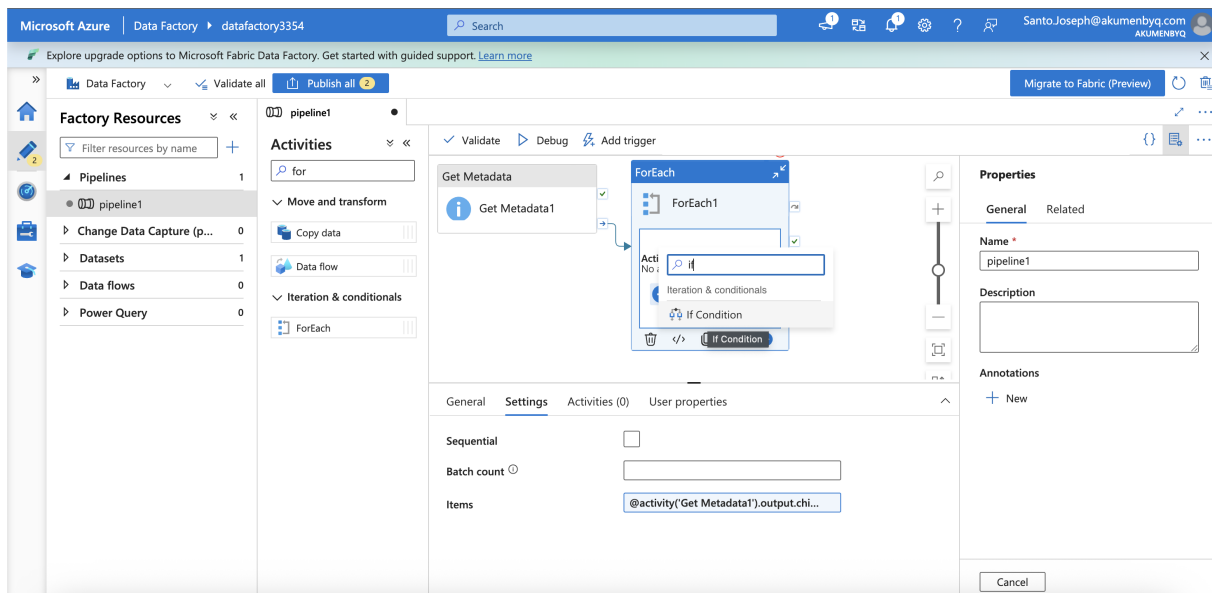
Opened the Pipeline expression builder for the ForEach Items field and selected "Get Metadata1 childItems", producing the expression `@activity('Get Metadata1').output.childItems`.



STEP 13

Add an If Condition inside the ForEach

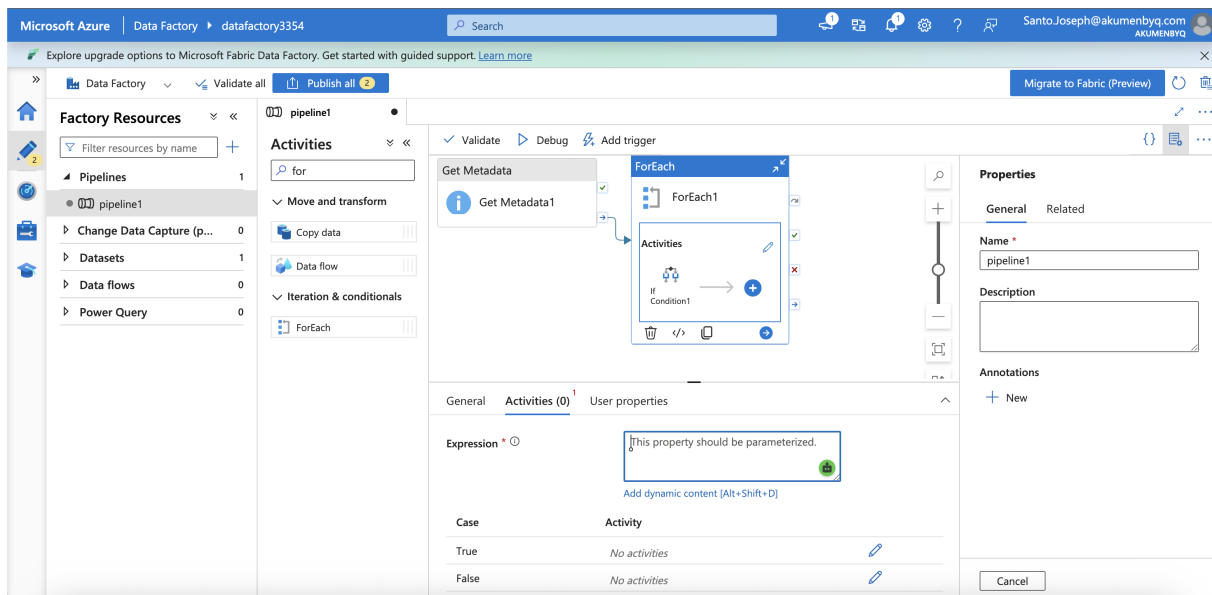
Inside the ForEach loop, searched for and added an "If Condition" activity to branch logic for each file being processed.



STEP 14

Set the If Condition expression

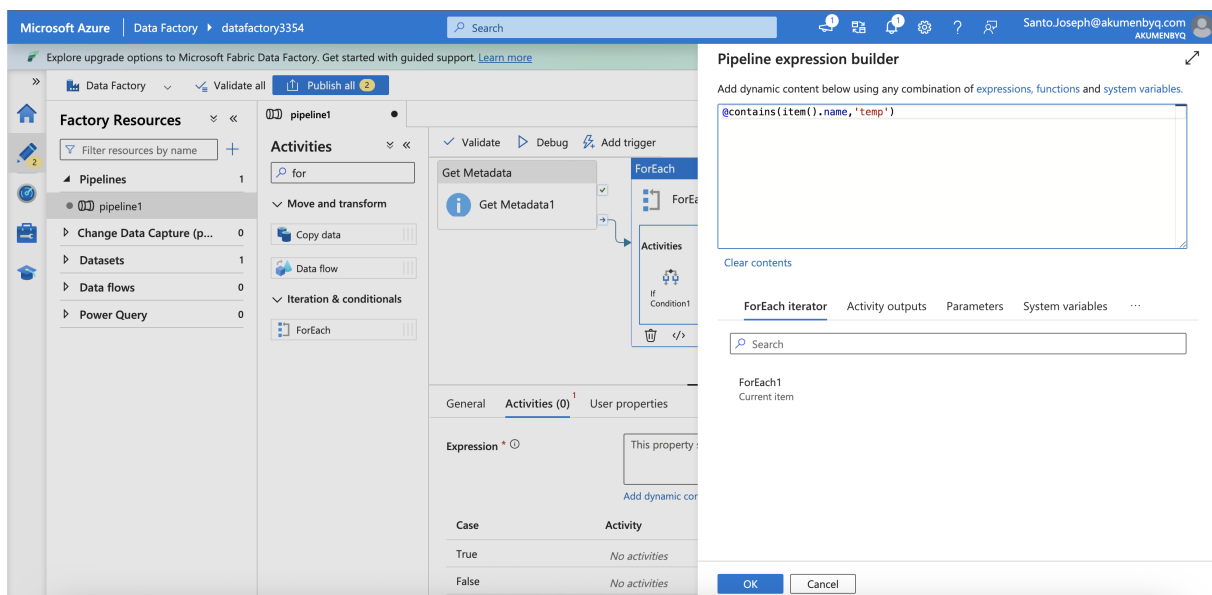
Opened the If Condition1 Activities tab and prepared to enter an expression that will be evaluated for every item in the loop.



STEP 15

Check file names using "contains"

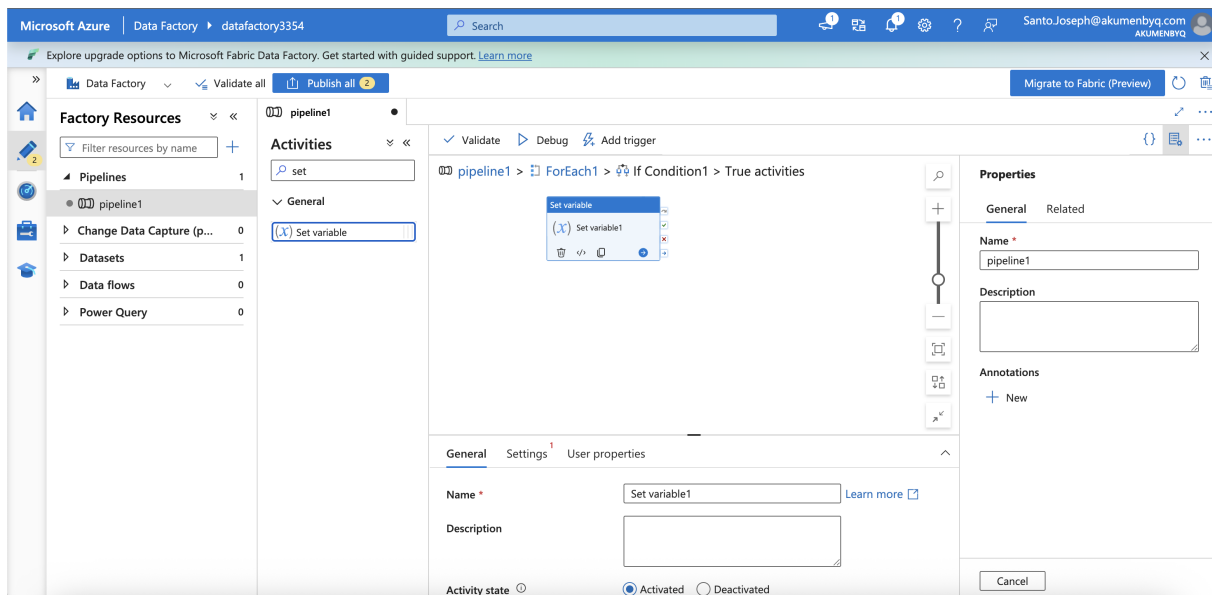
In the expression builder, used the ForEach iterator to build the condition `@contains(item().name,'temp')` — checking whether the current file's name contains "temp".



STEP 16

Add a Set Variable activity (True branch)

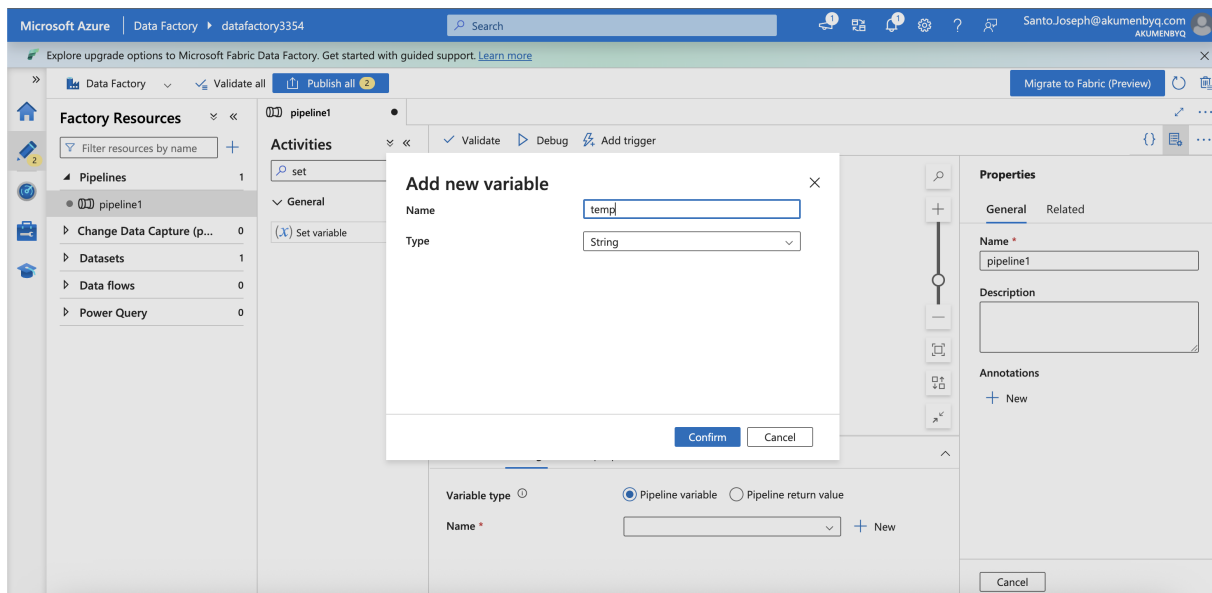
Inside the If Condition's True activities, added a "Set variable" activity to record which variable applies when the condition is true.



STEP 17

Create the "temp" pipeline variable

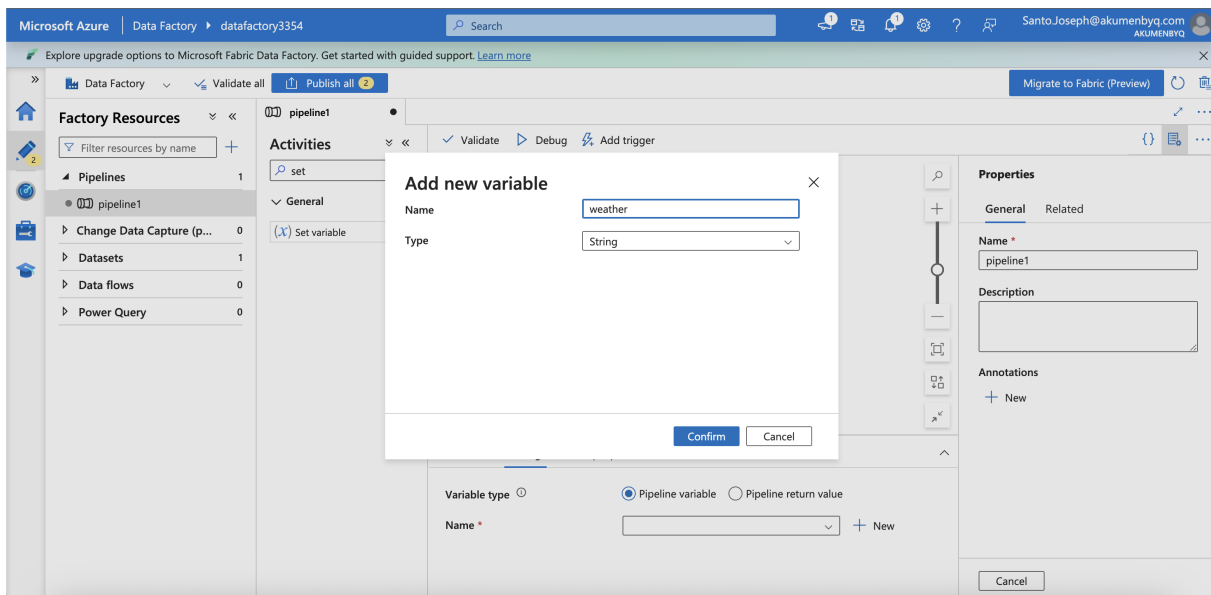
Created a new pipeline variable named "temp" of type String to be used by the Set Variable activity.



STEP 18

Create the "weather" pipeline variable

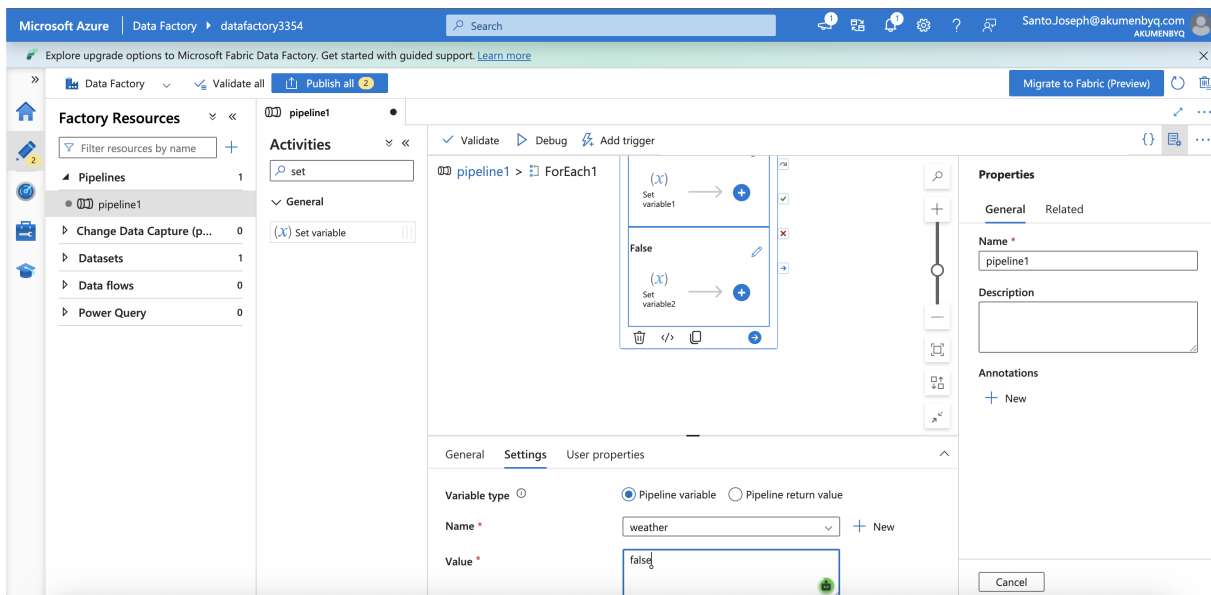
Created a second pipeline variable named "weather" of type String, for files that are not temperature files.



STEP 19

Set the value for the False branch

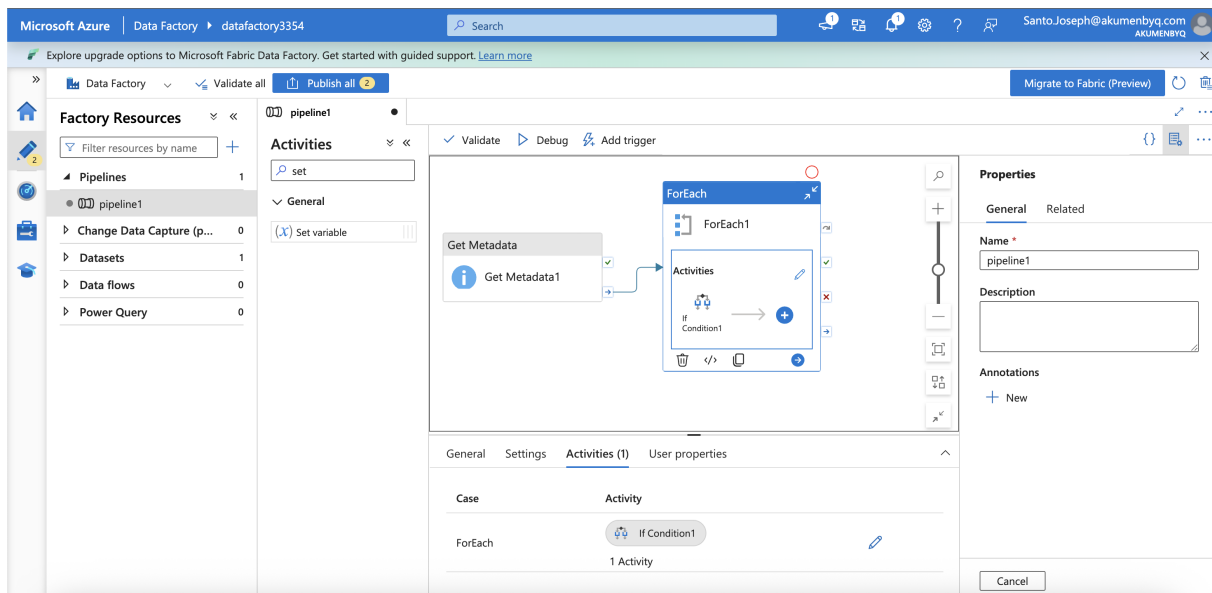
In the If Condition's False activities, added a second Set Variable activity, set its Name to "weather" and Value to "false".



STEP 20

Review the completed ForEach logic

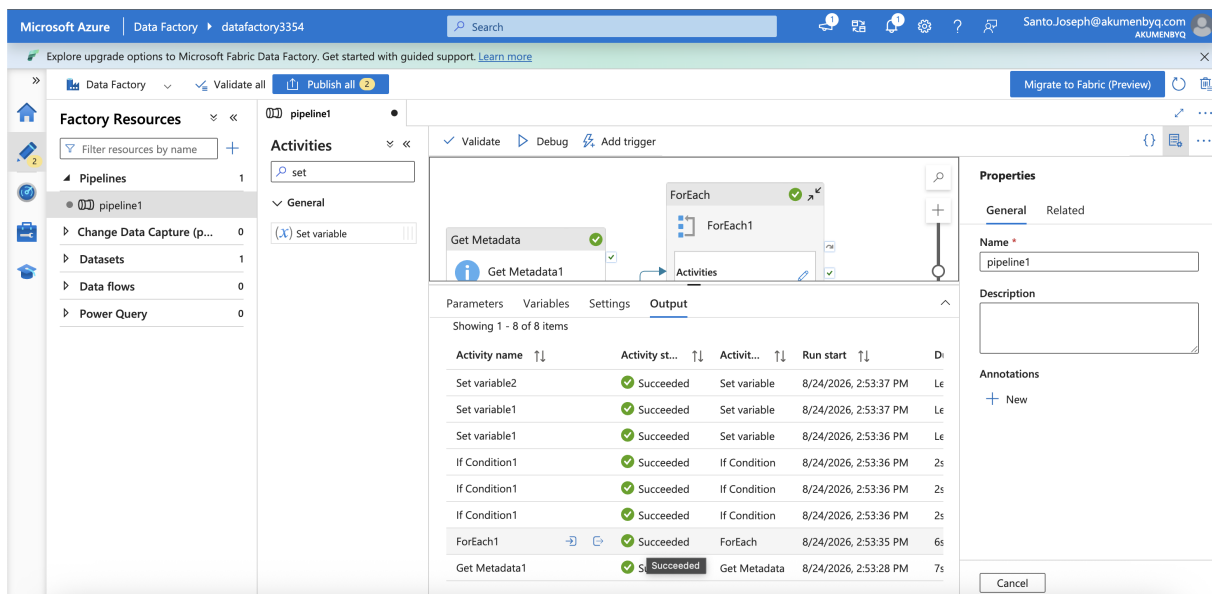
Final check of the ForEach activity: it now contains one If Condition (If Condition1) with a Set Variable activity in each branch, completing the loop-and-branch logic for the pipeline.



STEP 21

Debug the pipeline and check the Output

Ran the pipeline using Debug and opened the Output tab. All 8 activity runs succeeded: Get Metadata1, ForEach1, three If Condition1 runs (one per file), and matching Set variable runs — confirming the loop processed each file correctly.



STEP 22

Inspect Get Metadata1 output

Opened the Output for Get Metadata1 to confirm it correctly listed the childItems in the output folder, including temp_1.csv and temp_2.csv, each returned with name and type (File).

The screenshot shows the Microsoft Azure Data Factory interface for a pipeline named 'pipeline1'. The 'Activities' pane on the left shows 'Get Metadata1' selected. The 'Output' pane displays the following JSON structure:

```
{
  "childItems": [
    {
      "name": "temp_1.csv",
      "type": "File"
    },
    {
      "name": "temp_2.csv",
      "type": "File"
    }
  ]
}
```

The 'Properties' pane on the right shows the pipeline name 'pipeline1' and a description field. The 'Annotations' pane shows a 'New' button. The 'Run start' table at the bottom shows the pipeline run status.

Run start	Run end	Run status	Run duration
8/24/2026, 2:53:36 PM	Le		
8/24/2026, 2:53:36 PM	2s		
8/24/2026, 2:53:36 PM	2s		
8/24/2026, 2:53:36 PM	2s		
8/24/2026, 2:53:35 PM	6s		
8/24/2026, 2:53:28 PM	7s		

STEP 23

Check Set variable1 output (temp file match)

Opened the Output for Set variable1. For a file matching the "temp" condition, the variable "temp" was correctly set to "true".

The screenshot shows the Microsoft Azure Data Factory interface for a pipeline named 'pipeline1'. The 'Activities' pane on the left shows 'Set variable1' selected. The 'Output' pane displays the following JSON structure:

```
{
  "name": "temp",
  "value": "true"
}
```

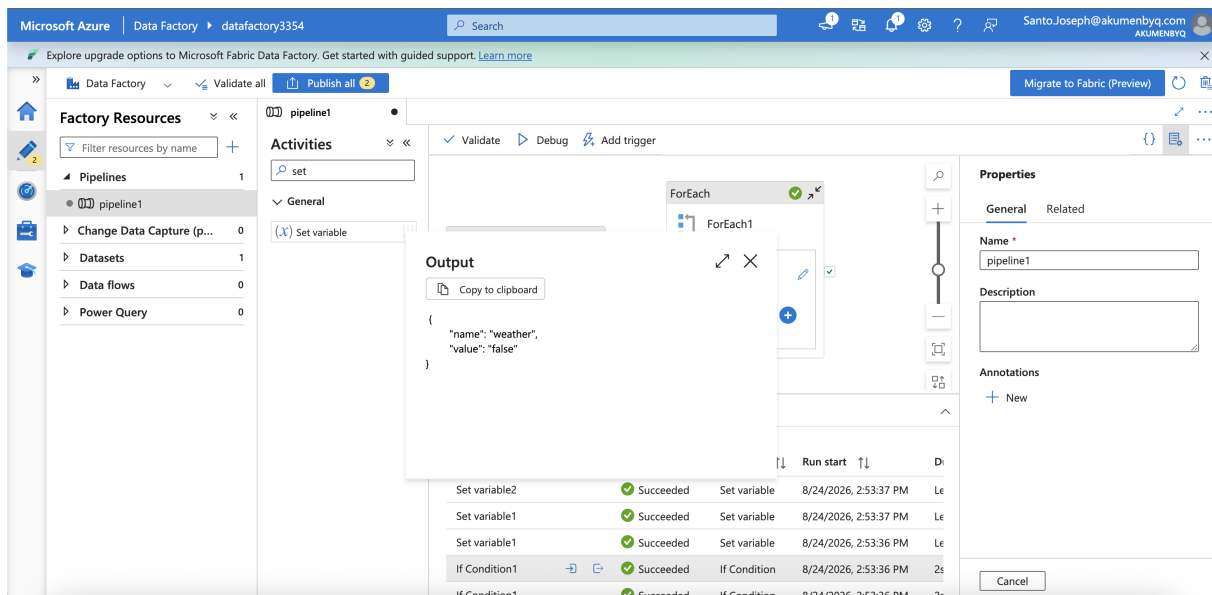
The 'Properties' pane on the right shows the pipeline name 'pipeline1' and a description field. The 'Annotations' pane shows a 'New' button. The 'Run start' table at the bottom shows the pipeline run status.

Run start	Run end	Run status	Run duration
8/24/2026, 2:53:37 PM	Le		
8/24/2026, 2:53:37 PM	Le		
8/24/2026, 2:53:36 PM	Le		
8/24/2026, 2:53:36 PM	2s		

STEP 24

Check the False-branch Set variable output

Opened the Output for the Set variable activity in the False branch. For a file that did not match the "temp" condition, the variable "weather" was correctly set to "false".



STEP 25

Confirm the True-branch output again

Re-checked another Set variable1 run in the True branch, again showing "name": "temp", "value": "true" — verifying the condition and variable logic works consistently across loop iterations.

